

Comparativo de Versões: Pilha de Navegação

UD-H1

Este documento apresenta uma análise comparativa detalhada entre a versão antiga do projeto (encontrada na pasta de backup `Navigation-main`) e a versão atualizada (presente na raiz do repositório ativo), evidenciando os avanços de engenharia de controle e integração com o ROS 2.

1. Visão Geral das Diferenças

A tabela abaixo resume as principais diferenças de arquitetura e parâmetros entre as duas versões:

Módulo / Funcionalidade	Versão Antiga (<code>Navigation-main</code>)	Versão Atualizada (Ativa / Raiz)	Impacto no Comportamento Real
Frequência de Controle	10 Hz (intervalo de 100ms)	50 Hz (intervalo de 20ms)	Correções de trajetória 5x mais rápidas, reduzindo desvios físicos repentinos.
Lógica do Controle (Arduino)	Termo Integral Puro (<code>kp = 4.0</code>) sem limites	PI + Feedforward com Anti-Windup	Elimina oscilações em "S" (<i>overshoot</i>) e estabiliza a velocidade das rodas sob carga.
Leitura de Encoders	<code>PinChangeInterrupt</code> (biblioteca lenta)	Interrupções de Hardware Nativas (<code>attachInterrupt</code>)	Garante contagem precisa de pulsos sem perda de ticks em altas velocidades.

TRIM de Motores	TRIM estático atenuando PWM final	TRIM Zerado (Compensação no PI/FF)	Evita o conflito cíclico entre atenuação de potência e malha de controle.
Rampas de Aceleração (ROS)	Inexistentes (comandos brutos diretos)	Rampas de Limite Linear (0.8 m/s^2) e Angular (1.5 rad/s^2)	Impede que as rodas do hoverboard patinem ou derrapem na partida/parada.
Sinais de Odometria (ROS)	Negação global ativada	Eixos e sinais normalizados com padrão ROS	Garante que o robô virtual no RViz2 se mova no mesmo sentido do robô físico.
Launch Files	Apontavam para o driver antigo com negações	Apontam para <code>base_driver_pulse</code>	Executa o driver sintonizado e calibrado por padrão.
Nó WASD Teleop	Ausente	Presente (<code>wasd_teleop.py</code>)	Permite teste de pista isolado de 1m sem necessidade de joysticks complexos.

2. Comparação Detalhada de Código

A. Firmware do Arduino (`odometrypulse-malha.ino`)

Abaixo estão os trechos críticos de código de controle de velocidade das duas versões:

Versão Antiga (`Navigation-main`):

```
// Frequência baixa (10 Hz)
if (dt_ms >= 100) {
    // Controle integral rudimentar sem limite (anti-windup)
    if (abs(setPointDir) < 0.001) pwmDir = 0;
    else pwmDir += (setPointDir - velDir) * kp; // kp = 4.0 atuando como acumulador de erro

    if (abs(setPointEsq) < 0.001) pwmEsq = 0;
    else pwmEsq += (setPointEsq - velEsq) * kp;

    // Interrupções não-nativas
    attachPinChangeInterrupt(digitalPinToPCINT(ODO_A_L), countEsq, RISING);
}
```

Versão Atualizada (Ativa):

```
// Frequência alta (50 Hz)
if (dt_ms >= 20) {
    // Erros de velocidade individuais
    float erroEsq = setPointEsq - velEsq;
    float erroDir = setPointDir - velDir;

    // Integral com anti-windup para evitar acúmulo excessivo de energia
    integralEsq = constrain(integralEsq + (erroEsq * (dt_ms * 0.001)), -20.0, 20.0);
    integralDir = constrain(integralDir + (erroDir * (dt_ms * 0.001)), -20.0, 20.0);

    // Controle PI real + Feedforward preditivo
    if (abs(setPointEsq) < 0.001) {
        integralEsq = 0;
        pwmEsq = 0;
    } else {
        float saidaEsq = (setPointEsq * feedForward) + (erroEsq * kp) + (integralEsq * ki);
        pwmEsq = constrain(saidaEsq, -100, 100);
    }

    // Interrupções diretas de hardware nativo nos pinos 2 e 3
    attachInterrupt(digitalPinToInterrupt(ODO_A_L), countEsq, RISING);
}
```

B. Driver ROS 2 Python (`base_driver_pulse.py`)

Abaixo estão os trechos onde os limites dinâmicos e sinais de odometria são calculados para evitar instabilidade:

Versão Antiga (`Navigation-main`):

```
def cmd_callback(self, msg):
    # Envio de comandos diretos sem rampa de aceleração
    v_left = v_linear - (v_angular * self.largura_eixo / 2.0)
    v_right = v_linear + (v_angular * self.largura_eixo / 2.0)

    # Ordem de pinos invertida
    comando = f"CMD:{v_right:.3f};{v_left:.3f}\n"

    # Inversão de sinais de odometria que gerava bugs no RViz/Nav2
    v_linear = -((v_dir + v_esq) / 2.0)
    v_angular = -((v_dir - v_esq) / self.largura_eixo)
```

Versão Atualizada (Ativa):

```
def cmd_callback(self, msg):
    # Cálculo dinâmico do intervalo
    now = self.get_clock().now()
    dt = (now - self.last_cmd_time).nanoseconds / 1e9
    self.last_cmd_time = now

    # Filtro de Aceleração Suave (Rampa linear e angular)
    target_v_linear = max(min(msg.linear.x, MAX_VEL_LINEAR), -MAX_VEL_LINEAR)
    dv_linear_max = MAX_ACC_LINEAR * dt
    dv_linear = target_v_linear - self.cmd_v_linear
    self.cmd_v_linear += max(min(dv_linear, dv_linear_max), -dv_linear_max)

    v_left = self.cmd_v_linear - (self.cmd_v_angular * self.largura_eixo / 2.0)
    v_right = self.cmd_v_linear + (self.cmd_v_angular * self.largura_eixo / 2.0)

    # Ordem correta dos motores (Esquerda;Direita)
    comando = f"CMD:{v_left:.3f};{v_right:.3f}\n"

    # Sinais de odometria normalizados com a direção física real do robô
    v_linear = ((v_dir + v_esq) / 2.0)
    v_angular = ((v_dir - v_esq) / self.largura_eixo)
```

3. Conclusão da Análise

A versão ativa na raiz do seu repositório resolve os problemas mecânicos e de localização por meio de: 1. **Filtro físico de aceleração no software (rampa)**, impedindo derrapagens. 2. **Alta taxa de atualização da malha fechada (50Hz)** para resposta instantânea a perturbações do solo. 3. **Controle PI + FF real**, garantindo torque no início do movimento e precisão na velocidade em regime permanente. 4. **Alinhamento correto de eixos e launch files**, de forma que ao rodar o comando principal o robô virtual corresponda perfeitamente ao robô real.

O backup `Navigation-main` deve ser mantido estritamente para histórico, não devendo ser compilado ou embarcado no robô de testes.